

DINO KI

Kann ein Netzwerk ein Spiel von selbst lernen?



Lernen durch Evolution statt durch Backpropagation: 50 Netze spielen parallel – die besten überleben und mutieren. Keine menschlichen Regeln nötig.

Laufen, Springen, Ducken – ganz ohne Spieler

LAUFEN

SPRINGEN

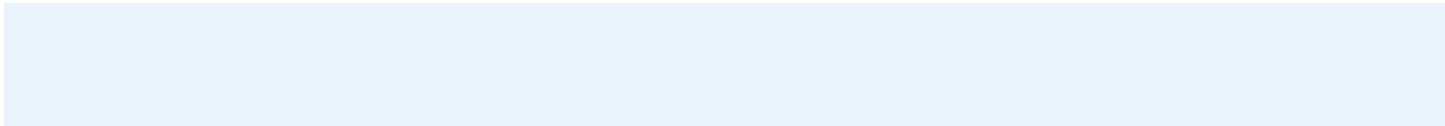
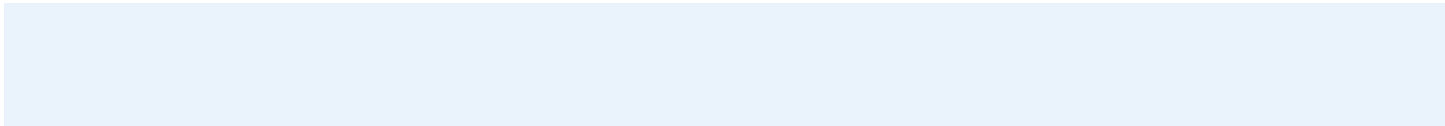
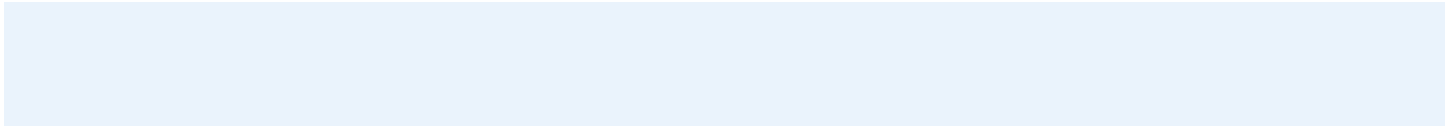
DUCKEN

PUNKTE

ENDE

Das Netz sieht nur die 6 Zahlen (Sensoren) – nicht das Bild.

Sechs Sensoren pro Frame – normalisiert auf 0 ... 1



Eingabe-Vektor

$$\mathbf{x} = (x_1, x_2, x_3, x_4, x_5, x_6)^T \in \mathbb{R}^6$$

x = Eingabe · T = Transponieren · $\mathbb{R}^6$ = 6 reelle Zahlen

Ausgabe des Netzes

$$\mathbf{y} = (y_1, y_2)^T \in \mathbb{R}^2$$

y_1 = SPRUNG · y_2 = DUCKEN

i6

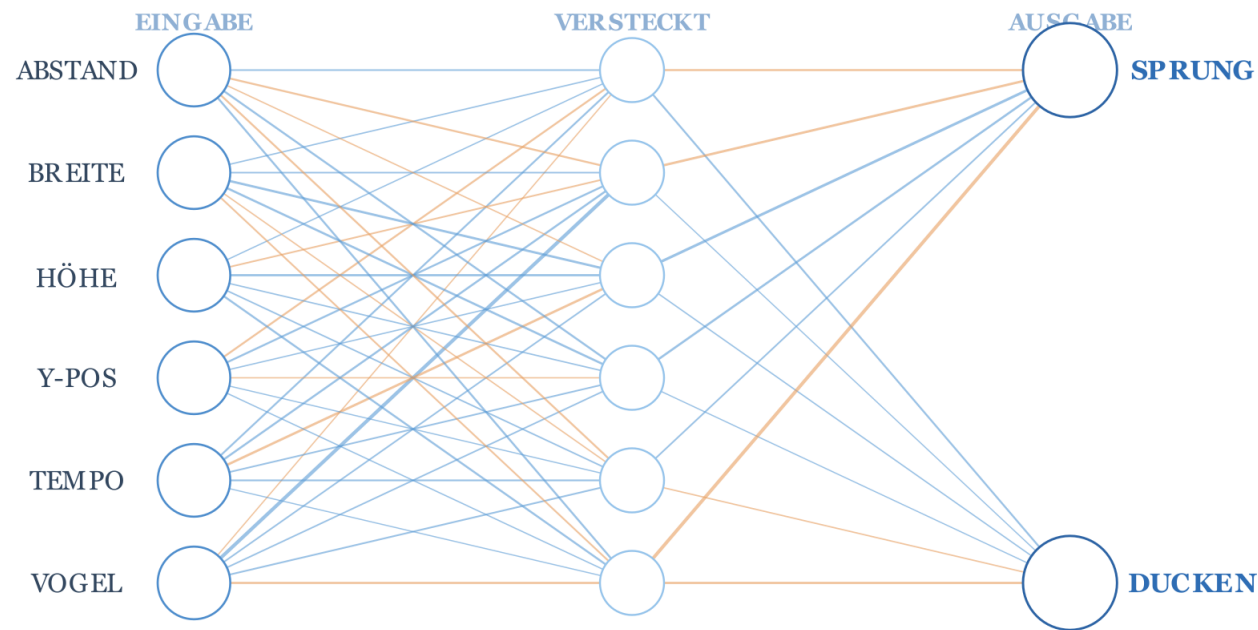
VOGEL

1 = Vogel · 0 = sonst

Fliegt ein Vogel an?

6 Eingaben □ 6 versteckte Neuronen □ 2 Ausgaben

DINO KI - NEURONALES NETZWERK



Blau = positives Gewicht · Orange = negatives Gewicht

6 Eingabe-Neuronen

6 Hidden-Neuronen

2 Ausgabe-Neuronen

48 Verbindungen (Start)

tanh-Aktivierung · Feed-Forward · die Topologie wächst mit jeder Generation.

Was passiert im Inneren eines Neurons?

$$z_j = \sum_{i=1}^n w_{ji} x_i + b_j$$

So liest man sie

Welche Bedeutung haben die Symbole?

Kurzform für ein Neuron mit 3 Eingängen:

$$z = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

Aus einer beliebigen Zahl wird ein nutzbares Signal

$$a_j = f(z_j)$$

$$f(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

So liest man sie

Warum tanh? Und welche Alternativen gibt es?

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$f(z) = \max(0, z)$$

Sigmoid $\rightarrow 0 \dots 1$ · ReLU $\rightarrow 0$ oder größer

Von y zur Aktion im Spiel

$y_1 > 0.5 \Rightarrow \text{SPRUNG},$

$y_2 > 0.5 \Rightarrow \text{DUCKEN}$

So liest man sie

$$z = 0.5 \cdot 0.8 + 0.3 \cdot 0.6 + 0.2 \cdot 0.4 + 0.86 = 1.52$$

Rechenbeispiel – von der Zahl zum Sprung

$$y_1 = \tanh(1.52) \approx 0.91$$

□ **0.91 > 0.5** □ **SPRUNG**

So bewertet die Evolution jedes Netz

$$F = 0.1 \text{ Frames} - 0.02 \text{ Ducks} - 1 \text{ Kollisionen}$$

So liest man sie

Rechenbeispiel

$$F = 0.1 \cdot 800 - 0.02 \cdot 20 - 1 \cdot 0 = 79.6$$

→ Je länger überlebt, desto höher die Fitness = „wie gut ist dieses Netz?“

+ 0.1 / Frame · Überleben lohnt sich

– 0.02 / Duck · Verhindert Dauerducken

– 1 / Kollision · Ende des Versuchs

Kleine Zufalls-Veränderungen an den Gewichten

$$w' = w + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2)$$

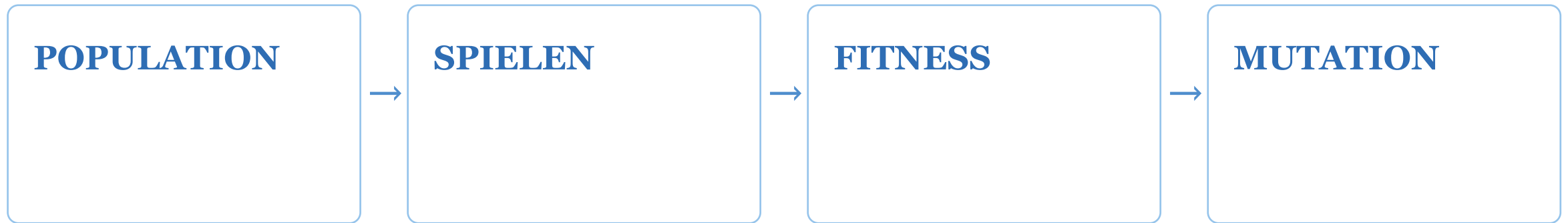
So liest man sie

Rechenbeispiel & Bedeutung

$$w' = 0.5 + 0.14 = 0.64$$

- ε klein ($\sigma = 0.5$) → wenig Zerstörung von gutem Wissen

NEAT: Netze spielen, die besten überleben und mutieren



Runde für Runde wiederholt sich der Zyklus – nach jeder Generation wird die nächste besser.



Kein Backpropagation, kein Loss, keine Lernrate – die Evolution selbst ist der Lern-Mechanismus.

Vom Zufallsnetz zum stabilen Spieler

NEAT-Besonderheit

Live im Dashboard

GEN 001

Zufall



GEN 010

erste Sprünge



GEN 025

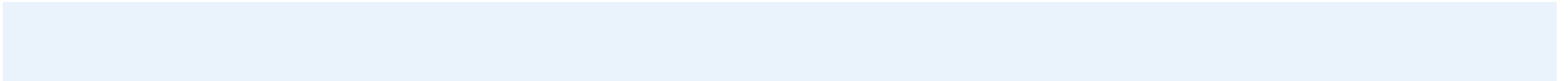
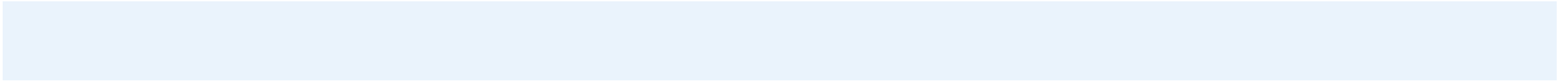
stabil



GEN 050

final

Lernfortschritt über die Generationen



50

Training im Überblick

finale Generation

Was lernt das Netz?

Python-Projekt, komplett lokal

pygame-ce

neat-python

pygame-menu

Manim

Danke für Ihre Aufmerksamkeit

$$z = \sum w_i x_i + b \rightarrow y = \tanh(z) \rightarrow y > 0.5 \Rightarrow \text{Aktion}$$

Die ganze KI in einer Zeile